SysMonitor

System Design Description

Abstract

This document describes the design and implementation of the SysMonitor support system of Arrowhead 5.2.2 (`ah5-SysMonitor-java-spring`).

Contents

1 Overview **3**

2 Implementation **4**
2.1 Implementation language and tools 4
2.2 Functional properties implementation 4
2.3 Non functional properties implementation 5
2.4 Deployment 6
2.5 Local modifications 6
2.6 Verification 6

3 Services **7**

4 References **7**

5 Revision History **9**
5.1 Amendments 9
5.2 Quality Assurance 9

1 Overview

This document describes the SysMonitor system implementation, which realizes the SysMonitor SysD [1]. SysMonitor collects the native `sysMonitor` observations of all systems from the AMQP broker, stores their history and republishes them on the MQTT broker (`sysMonitorData`). In Section 2, we describe implementation details of the system.

2 Implementation

This implementation realizes the SysMonitor SysD in `ah5-SysMonitor-java-spring` on the Arrowhead Java Spring pattern (as Blacklist and EventHandler). The Java sources, the private copy of the Arrowhead common library (`vendor/`), the tests, the image and the deployment definition are owned by this folder. The previous implementations are preserved in `legacy/` and are excluded from the build.

2.1 Implementation language and tools

- Java 21, Spring Boot 3.4 (web, security, data JPA, scheduling) with the vendored Arrowhead common library (`common-utils`, `data-transfer-objects`); RabbitMQ AMQP client for the intake and Eclipse Paho MQTT v5 for the output. Maven produces the Spring Boot JAR `arrowhead-sysmonitor-5.2.2.jar` with the entry point `eu.arrowhead.sysmonitor.SysMonitorMain`.
- Storage: the private MySQL schema `ah_sysmonitor` through JPA (`ddl-auto=validate`), reached with the MariaDB driver and `sslMode=verify-full`. The application account has data-manipulation privileges only.

2.2 Functional properties implementation

The following components (package `eu.arrowhead.sysmonitor`) implement the system:

Component	Responsibility
`SysMonitorSystemInfo`	System and service models registered by the common `ApplicationInitListener`: `sysMonitorManagement`, `sysMonitorHistory`, `sysMonitorData` (`generic_mqtts`) and `generalManagement`. The common `SysMonitorPublisher` registers and publishes the system's own `sysMonitor` observations.
`api.http.HistoryAPI`, `ManagementAPI`, `GeneralManagementAPI`, `HealthAPI`	The HTTP paths of the native 5.2.2 implementation: `/sysmonitor/history/*`, `/sysmonitor/management/*`, `/sysmonitor/mgmt/general/status` (plus `get-log` and `get-config`) and `/health`, port 8475.
`service.AmqpIntake`	Durable queue `SysMonitor.observations` on the `arrowhead` topic exchange; validates each observation against its routing key, stores it, then acknowledges (rejects invalid ones, requeues on storage failure).
`service.ObservationFormat`	Normalised observation envelope and derived observation identifier (SHA-256 of routing key and canonical JSON), unchanged from the native implementation.
`service.MonitoringService`	Orchestrates the authorized `sysMonitor` providers (`generic_amqps`, `CERT_AUTH`) and applies the bindings derived from the subscriptions or the default subscription.
`service.MqttOutput`	MQTT v5 output `arrowhead/<cloud>/sysmonitor/data/<type>/<provider>`, QoS 1, from the outbox.
`service.SysMonitorJobs`	Scheduled broker (re)connection, orchestration (`ORCHESTRATION_INTERVAL_SECONDS`), outbox (1 s) and retention prune (`SYSMONITOR_PRUNE_INTERVAL_SECONDS`).
`jpa.service.MonitorDbService`	Store of subscriptions (revisions, `If-Match`), bindings, outbox, history and audit.

Every request passes the Spring filter chain: the common `OutsourcedFilter` (identity token verified with Authentication and bound to the client certificate), `CallerProfileFilter` (certificate profile `sy` or `op`), `SysMonitorAccessFilter` (`sysMonitorHistory` operations decided by ConsumerAuthorization, cached 30 s when granted; every other `/sysmonitor` path only for LocalCloudManager or Sysop) and the common management filter for `/sysmonitor/mgmt/` (Sysop and the whitelist LocalCloudManager). Errors use the Arrowhead error body (`errorMessage`, `errorCode`, `exceptionType`, `origin`) with the status codes of the native implementation.

2.2.1 Data handled and stored

Private MySQL schema `ah_sysmonitor` (`deployment/database/init.sql`, `src/main/resources/database`):

Tables	Content
`observation, outbox`	History (retention-bounded, `SYSMONITOR_RETENTION_DAYS`) and MQTT output queue.
`subscription, amqp_binding, audit`	Subscriptions, broker bindings and audit log.
`logs`	Log4j2 JDBC log of the common library (`get-log`).

The interfaces exchange JSON objects built at run time; their structure is specified in the IDD documents. The history of the former SQLite store is not migrated; it is retention-bounded.

2.3 Non functional properties implementation

2.3.1 Security

The HTTPS API requires mutual TLS 1.2/1.3 with the PKCS12 identity and trust stores (`client-auth=need`); client certificates are checked against the CRL of the cloud CA (`arrowhead.ssl.crl-file`). The AMQP intake and the MQTT output use the same system certificate (AMQP SASL EXTERNAL), verify the broker host name and check the broker certificate against the same CRL. The common `SecurityProfileGuard` refuses to start with TLS off, declared authentication, `service.authorization.enabled=false` or plain broker connections, unless `arrowhead.security.profile=development` is selected for an isolated test.

2.3.2 Power management

None; the system is a server process without power states. The `sysMonitor` `power` observation always reports line supply.

2.3.3 Internal monitoring

Logs go to stdout, a file under `/app/logs` and the `logs` table. The common `SysMonitorPublisher` publishes the system's own `sysMonitor` observations every 30 s; its health is `alive` only with both broker connections.

2.3.4 Configuration

`config/application.properties` holds the deployment settings; the environment names `LOCAL_CLOUD`, `ORGANIZATION`, `AMQP_HOST/PORT`, `MQTT_HOST/PORT`, `SYSMONITOR_*`, `ORCHESTRATION_INTERVAL_SECONDS` and `CRL_FILE` remain placeholders with defaults. Secrets (store, identity and database passwords) are read from `/run/secrets` (Spring config tree).

2.4 Deployment

The local `Dockerfile` builds only this folder and runs `arrowhead-sysmonitor-standalone:5.2.2`. The local `compose.yaml` joins an operator-selected existing cloud network and adds the private `sysmonitor-db` (MySQL 8.4.11 with mandatory TLS on an internal network, schema and data-manipulation-only account created by `deployment/database`). The HTTPS API uses port 8475. The process runs as UID 10001 with a read-only root filesystem; only `/tmp` and `/app/logs` are writable tmpfs. Key and trust stores, the database CA and the CRL are mounted individually and read-only. This packaging is supplied for operator deployment; existing test-cloud deployments are separate deployment consumers and are not changed by it.

To select this system, copy `.env.example` to `.env`, provision its identity/password, PKCS12 key and trust stores, CRL, database TLS material and passwords, configure network and registry, then run `docker compose up -d --build` from this folder. The database volume must never be shared with another system. Stopping this Compose project preserves its volume and does not stop other systems.

ServiceRegistry is the bootstrap endpoint. Authentication and DynamicServiceOrchestration are looked up there; ConsumerAuthorization and the monitored `sysMonitor` providers are orchestrated through DSO and require the cloud's authorization rules. The AMQP broker and the MQTT v5 output broker must be available for operational readiness: SysMonitor reports degraded (HTTP 503 on `/health` and `generalManagement` status) when AMQP intake or MQTT output is disconnected.

2.4.1 Ownership and build boundary

The common library is a private vendored source copy (`vendor/PROVENANCE.json`); there is no shared runtime process, data directory or Maven parent other than Spring Boot. Security fixes to the common library must be propagated to every vendored copy. Build with Java 21 and `mvn -B clean verify` (`scripts/verify.sh` also checks the artifact contents), or use the local Docker build; `scripts/verify-image.sh` checks that the image runs read-only and refuses insecure settings.

2.5 Local modifications

Notes on this implementation in the workspace:

- Native Java port (2026-10-16); bindings re-applied on every refresh (2026-10-19).
- Converted to the Arrowhead Java Spring pattern with a private MySQL schema (2026-11-08). HTTP paths, status codes, AMQP routing keys, the observation format and the MQTT topics and payloads are unchanged; error bodies now use the Arrowhead error format, `/health` requires the identity token like every other path, and `generalManagement` also offers `get-log` and `get-config`.

2.6 Verification

The project's JUnit tests cover the observation format and identity binding, the store (deduplication, outbox, revisions, filters, prune), orchestration and binding selection, the intake acknowledgement rules, the ConsumerAuthorization cache, the secure-profile refusal and a route contract test of every previously served path and method behind the deployed filter chain (unauthenticated, wrong certificate and authorised). Opt-in tests run the whole application against a fresh MySQL container built from `deployment/database` (schema validation, data-manipulation-only grants) and an AMQP to store to MQTT round trip against local RabbitMQ and Mosquitto with mutual TLS. A live Go/Rust federation run is not part of this verification.

3 Services

The implementation of the services is based on the SD and IDD documents referenced below.

Services produced	**SysD ref**	**SD ref**	**IDD ref**
`sysMonitorManagement`	[1]	[2]	[3]
`sysMonitorHistory`	[1]	[4]	[5]
`sysMonitorData`	[1]	[6]	[7]
`sysMonitor`	[1]	[8]	[9]
Services consumed	**SysD ref**	**SD ref**	**IDD ref**
serviceDiscovery	[10]	[11]	–
identity	[12]	[13]	–
serviceOrchestration	[14]	[15]	–
authorization	[16]	[17]	–
sysMonitor (all providers)	AMQP broker	–	–
sysMonitorData	MQTT broker	–	–

4 References

[1] J. Delsing, “SysMonitor – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-SysMonitor-java-spring/documentation/v5.2.2/SysMonitor-v5.2.2-SysD.pdf.

[2] ——, “sysMonitorManagement – Service Description (SysMonitor),” Eclipse Arrowhead, SD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorManagement-SD.pdf.

[3] ——, “sysMonitorManagement HTTP/TLS/JSON – Interface Design Description (SysMonitor),” Eclipse Arrowhead, IDD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorManagement-HTTP-IDD.pdf.

[4] ——, “sysMonitorHistory – Service Description (SysMonitor),” Eclipse Arrowhead, SD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorHistory-SD.pdf.

[5] ——, “sysMonitorHistory HTTP/TLS/JSON – Interface Design Description (SysMonitor),” Eclipse Arrowhead, IDD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorHistory-HTTP-IDD.pdf.

[6] ——, “sysMonitorData – Service Description (SysMonitor),” Eclipse Arrowhead, SD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorData-SD.pdf.

[7] ——, “sysMonitorData MQTT/TLS/JSON – Interface Design Description (SysMonitor),” Eclipse Arrowhead, IDD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorData-MQTT-IDD.pdf.

[8] ——, “sysMonitor – Service Description (SysMonitor),” Eclipse Arrowhead, SD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitor-SD.pdf.

[9] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (SysMonitor),” Eclipse Arrowhead, IDD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitor-AMQP-IDD.pdf.

[10] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-SysD.pdf.

[11] ——, “serviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-serviceDiscovery-SD.pdf.

[12] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-SysD.pdf.

[13] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-identity-SD.pdf.

[14] ——, “DynamicServiceOrchestration – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-SysD.pdf.

[15] ——, “serviceOrchestration – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-serviceOrchestration-SD.pdf.

[16] ——, “ConsumerAuthorization – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-SysD.pdf.

[17] ——, “authorization – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-authorization-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-25	5.2.2	Independent Java build, private helpers and state, per-system deployment and configuration.	Codex (draft)
3	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated.	Claude Opus 5.5 (draft)
4	2026-11-08	5.2.2	Implementation converted to the Arrowhead Java Spring pattern with a private MySQL schema; implementation, security, deployment and verification sections updated.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	